



Bachelor study programme
Medical and Bioinformatics
A-4232 Hagenberg, Austria

A population-based multi-agent approach for simulating large-scale system dynamics in discrete environments

Bachelor thesis

to obtain the academic degree of
Bachelor of Science in Engineering

Submitted by

Tim Peko

Reviewed by FH-Prof. PD DI Dr. Stephan Winkler

Hagenberg, October 2026

Declaration

I hereby declare and confirm that this thesis is entirely the result of my own original work. Where other sources of information have been used, they have been indicated as such and properly acknowledged. I further declare that this or similar work has not been submitted for credit elsewhere. This printed copy is identical to the submitted electronic version.

Date

Signature

Acknowledgment

Special thanks to Kurzgesagt for inspiring the thesis' topic in their video "Why Earth Sucks Compared to the Planet Hestia".

Abstract

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat.

Kurzfassung

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat.

Contents

Declaration	ii
Acknowledgment	iii
Abstract	iv
Kurzfassung	v
1 Introduction	1
1.1 Motivation	1
1.2 Research Questions	3
1.3 Goals	3
1.4 Hypothesis	4
2 Foundational works	5
2.1 Algorithms	5
2.2 Frameworks	5
2.3 Tierra	5
2.4 Avida	5
2.5 Räumlich erweitertes TaNa Modell	5
2.6 Multi-Level Makroevolutions-Frameworks	5
2.7 klassische ABMs	5
2.8 Homeostatic Reinforcement Learning	6
2.9 Lotka-Volterra-Dynamiken	6
2.10 Das “EcoTwin”-Projekt	6
2.11 “PredPreyGrass” via MADRL (CoMSES Net Repositorium)	6
2.12 Multi-Agent Inverse Reinforcement Learning	7
2.13 Agent-Based Modeling vs. Population-Based Modeling	7
3 Methodology	9
3.1 Environment Simulation	9
3.1.1 Cellular Automata	9
3.2 Agent Representation	9

3.2.1	Definition of a single species	10
3.3	Approaches to Agent Control	10
3.3.1	Skipped: Custom Reinforcement Learning Model	10
3.3.2	Used Instead: Open-Source Typesafe Decision Models (“Jev”-Style)	10
3.3.3	Species -> Environment: Hard Rule-Based	10
3.3.4	Secondary Research Question: Applicability of Generalistic Decision Models	11
3.4	Self-Supervised MARL	11
3.5	Vom trainierten Modell zur Laufzeitformel	12
4	Rust Implementation	13
4.1	Crate structure	13
4.2	Species registry	14
4.3	Tick pipeline	14
4.4	Determinism rules	14
4.5	Population behavior and migration	15
4.6	Species registry as the behavior interface	15
4.7	Evolution as a kernel phase	15
4.8	Closed-form behavior surrogate	16
4.9	Verification and benchmarks	16
4.10	Extensibility contract	17
4.11	Diffusion cellular automaton	17
5	Results	18
5.1	Performance	18
5.2	Accuracy	18
5.3	Comparison	18
6	Conclusion	19
6.1	Limitations	19
6.2	Future Work	19
A	External Resources	20
B	Source Codes	21
B.1	Snippets	21
C	Abbreviations	22

References	23
List of tables	24

Chapter 1

Introduction

1.1 Motivation

When we look at nature, we see an incredibly complex interplay of countless animal and plant species. They prey on one another, compete for space, adapt to temperatures, and change the world around them simply by existing. This process, known as evolution, has shaped our Earth over billions of years. However, modern science faces a fundamental problem: we cannot put an entire planet into a laboratory. If we want to understand how a lifeless rock turned into a green, oxygen-rich world full of life, we run into an issue of time. Real evolution is far too slow to be observed in real-time.

This is exactly where modern bioinformatics comes into play. The overarching goal of this research field is to move evolution into the computer. If we can program realistic virtual worlds, we can simulate millions of years of Earth's history in a single afternoon. This is not just a theoretical game; it has enormous practical value for the real world. By understanding the fundamental rules of how entire ecosystems build up or collapse, we can better predict how our modern environment will react to dramatic changes—whether caused by global climate change, invasive species, or pollution. We are essentially creating a “digital twin” of ecological networks to test the breaking points of life.

This thesis focuses on developing exactly this kind of planetary simulation. The goal is to build a computational framework that can simulate the development of species on a global scale. However, instead of trying to calculate every single animal or every leaf on a tree—which would immediately overload even the most powerful supercomputers—this work introduces a clever, alternative approach. The virtual world is divided into a massive spatial grid. The cells in this grid do not contain individual animals, but rather entire populations of species acting as agents. If a species thrives, its “influence” in that region grows. If it gets too crowded or the climate becomes too harsh, a part of the population migrates to neighboring cells.

A central element of this simulation is the realization that life does not merely adapt to its environment; it actively rebuilds it. A famous example from Earth's history, which is modeled in this work, is the Great Oxidation Event. Early plant-like life forms produced massive amounts of oxygen. What is vital for us today was pure poison for the anaerobic world back then. By simply spreading, these plants changed the chemical composition of the entire atmosphere. This creates enormous evolutionary pressure: other species must adapt, flee, or go extinct. Such massive, global feedback loops between life and the environment will be realistically modeled in the system developed here.

Furthermore, this thesis addresses a highly fascinating core question: What actually happens when life has it “too easy”? Our hypothesis is that under perfect global living conditions—meaning it is warm everywhere, there is plenty of food, and there are no extreme environmental barriers—the absolute biodiversity on the planet paradoxically decreases rather than increases. If nature does not force species to specialize in certain harsh niches, so-called “generalists” emerge. These are super-species that can survive almost anywhere. They spread unhindered across the globe and ultimately outcompete all other, weaker species. While there might be many animals in one specific location, looking at the planet as a whole, every region looks exactly the same. This theory of biological homogenization will be put to the test through the simulation.

To realize this ambitious project, standard programming is not enough. The simulation is written in the modern, high-performance programming language Rust. Additionally, to ensure that the computer knows how the millions of populations should behave realistically, techniques from the field of artificial intelligence are utilized: instead of training a custom reinforcement learning model, the interaction decisions between populations—how a population reacts to its environment and to other species—are delegated to small, open-source decision models of the “Jev” type, which answer typed questions with calibrated probabilities in a single forward pass. The opposite direction, in which populations actively reshape their environment, is deliberately kept strictly rule-based.

In summary, this thesis combines biology, environmental physics, and cutting-edge software engineering. It provides a computational tool to make the complex, often invisible gears of macroevolution tangible, helping us understand why our planet's biodiversity looks the way it does today—and how quickly it can collapse when the rules of nature change.

1.2 Research Questions

RQ1 **How can species-to-species and environment-to-species interaction be approximated at scale without training a custom reinforcement learning model?**
Research Question 1

Instead of training a bespoke *RL* policy, population-level interaction decisions are answered by small, open-source, typesafe decision models (so-called “System 1” decision models, e.g. JevK5 or a fine-tuned variant of Laya). These models receive the state of a cell and a set of typed questions (yes/no, choice, score) and return calibrated probabilities in a single forward pass, without generating text.

RQ2 **How can the reverse direction, species-to-environment interaction, be captured?**

Species actively reshape their environment (e.g. oxygen production during the Great Oxidation Event). This direction is deliberately modeled as hard, deterministic rule sets—to be defined—rather than learned behavior.

RQ3 **How far do such generalistic decision models transfer to ecological interaction decisions, and what does their behavior reveal about the biological or systems-level understanding these models have acquired?**
Research Question 3

A secondary, exploratory question: since these models are trained on general-purpose decision data rather than ecological domain data, their applicability in this domain is not self-evident. Deviations and failure patterns are of independent interest, as they hint at what high-level biological or systems understanding such generalistic models actually possess.

1.3 Goals

- Development of a scalable, grid-based planetary simulation in Rust. The environment is updated by physical rules, species populations act as agents.

- Approximation of species-to-species and environment-to-species interaction via open-source, typesafe decision models (JevK5, fine-tuned Laya) instead of a custom *RL* model.
- Hard rule-based modeling of species-to-environment interaction.
- Evaluation of whether the simulation reproduces plausible macroevolutionary dynamics (e.g. the Great Oxidation Event) and the biodiversity homogenization hypothesis.

1.4 Hypothesis

α -Diversity $\sim$ γ -Diversity

Additionally, as a secondary expectation tied to RQ3: the generalistic decision models will be applicable to ecological interaction decisions only to a limited degree—expect transfer gaps and characteristic failure patterns that reveal how much (or how little) high-level biological and systems understanding such models actually encode.

Chapter 2

Foundational works

2.1 Algorithms

2.2 Frameworks

2.3 Tierra

[https://en.wikipedia.org/wiki/Tierra_\(computer_simulation\)](https://en.wikipedia.org/wiki/Tierra_(computer_simulation))

2.4 Avida

Inspired by Tierra (Sec. 2.3)

<https://de.wikipedia.org/wiki/Avida>

2.5 Räumlich erweitertes *TaNa* Modell

2.6 Multi-Level Makroevolutions-Frameworks

State of the Art, Bottom-Up Ansätze. 2025 von Latorre et al. in PLoS ONE. (Latorre et al., 2025)

2.7 klassische *ABMs*

Oft nicht skalierbar

CoMSES Net (OpenABM)

2.8 Homeostatic Reinforcement Learning

https://www.researchgate.net/publication/354597646_Continuous_Homeostatic_Reinforcement_Learning_for_Self-Regulated_Autonomous_Agents (Laurençon et al., 2021)

2.9 Lotka-Volterra-Dynamiken

2.10 Das “EcoTwin”-Projekt

Twin models of natural ecosystems hold great promise for informing real-world decisions on sustainable land use and biodiversity conservation. However, existing simulations of animal behavior often rely on manually crafted rules, limiting their scalability and practical utility. Here, we present a flexible and scalable agent-based modeling approach that uses reinforcement learning—instead of hand-coded rules—to simulate animal behavior. We validate this approach across ten alpine ecosystems featuring wolves, chamois, and vegetation. By comparing model outputs with empirical data, we show that the simulations reproduce realistic ecological and behavioral patterns, including population dynamics, life history traits, and social interactions. We then use the model to assess ecosystem resilience under scenarios of habitat degradation, game hunting, and heat stress. Our framework paves the way for realistic simulations advancing our ability to predict ecosystem responses to disturbance and tipping points leading to biodiversity loss, in order to support conservation planning and guide the sustainable use of natural resources.

(Strannegård et al., 2025)

2.11 “PredPreyGrass” via MADRL (CoMSES Net Repository)

Exploring learned cooperation, coevolution and free-riding. Learning is achieved through Multi-Agent Deep Reinforcement Learning (MADRL) in an ecological environment. The environment emits no other than sparse reproduction rewards. No reward shaping, no explicit cooperation signal. (HBP1969, 2026)

2.12 Multi-Agent Inverse Reinforcement Learning

Recent comparative studies between classical population-based modeling (PBM)—which typically uses differential equations—and agent-based modeling (*ABM*) have increasingly applied Multi-Agent Inverse Reinforcement Learning (MIRL). Instead of prescribing a reward function to the agents, MIRL uses empirical macroscopic data to infer, in reverse, the underlying reward functions that drive real-world collective or dispersal behaviors.

These studies provide mathematical evidence that *ABMs*, when applied to very large and homogeneous populations, can accurately approximate the same macroscopic dynamics as traditional PBM approaches. This finding directly supports the design choice of this thesis: modeling agents as aggregated population units, rather than simulating millions of individual entities, is both justified and efficient for capturing large-scale ecological dynamics.

2.13 Agent-Based Modeling vs. Population-Based Modeling

This paper addresses comparative evaluation of population-based simulation in comparison to agent-based simulation for different numbers of agents. Population-based simulation, such as for example in the classical approaches to predator-prey modelling and modelling of epidemics, has computational advantages over agent-based modelling with large numbers of agents. Therefore the latter approaches can be considered useful only when the results are expected to deviate from the results of population-based simulation, and are considered more realistic. However, there is sometimes also a silent assumption that for larger numbers of agents, agent-based simulations approximate population-based simulations, which would indicate that agent-based simulation just can be replaced by population-based simulation. The paper evaluates such assumptions by two detailed comparative case studies: one in epidemics, and one in economical context. The former case study addresses the spread of an infectious disease over a population. The latter case study addresses the interplay between individual greed as a psychological concept and global economical concepts. It is shown that under certain conditions agent-based and population-based simulations may show similar results, but not always. ¹ Parts of this paper have been presented at the conferences ECMS'08 [20] and IEA/AIE'10 [7]. The current paper extends these conference papers by providing additional simulation experiments, simulation results relating to

empirical data, more detailed analyses of simulation results, and a more extensive discussion of related work and of the differences between agent-based and population-based modelling.

(Bosse et al., 2012)

Grundlage für den Ansatz, dass Populationbasierte Modelle für sehr große und homogene Populationen exakt das gleiche makroskopische Verhalten wie traditionelle PBMs approximieren können.

Chapter 3

Methodology

3.1 Environment Simulation

Eucledian simulation (discrete timesteps) Grid-based, every cell has environment state

- Gases amounts
 - O_2
 - CO_2
 - CH_4
 - H_2O
 - ...
- Temperature
- Light
- ...

There are global environment parameters such as:

- Gravity
- Length of day and night
- Axis tilt
- ...

3.1.1 Cellular Automata

Every cell's environment state is updated based on the state of the neighboring cells.
Pressure equalization is simulated by diffusion of the gases.

3.2 Agent Representation

Every agent represents a single species/population. It has different attributes:

- ...

A cell holds a list of species and their amounts present in the cell.

3.2.1 Definition of a single species

Evolutionary lineage with similar enough traits to meaningfully differ from other species.

3.3 Approaches to Agent Control

How to update the population's attributes, specifically amount present and evolution over time?

3.3.1 Skipped: Custom Reinforcement Learning Model

Training a custom reinforcement learning model for predicting species species and environment -> species interactions was considered, but is deliberately **skipped** in this work:

- Training a custom *RL* model requires a reward signal that does not naturally exist for ecological interactions.
- Curricula and simulation loops for training would dominate the implementation effort, while the actual research focus lies on the large-scale system dynamics.
- Pretrained open-source models offer a functional alternative at a fraction of the cost.

3.3.2 Used Instead: Open-Source Typesafe Decision Models (“Jev”-Style)

Species species and environment -> species interactions are instead answered by small, open-source, typesafe “System 1” decision models, e.g.:

- JevK5: open-weight model (Apache-2.0), answers typed yes/no, choice, and score questions with calibrated probabilities in a single forward pass, without generating text.
- Laya: open-source non-autoregressive decision model; the base checkpoint is intended as a foundation to be fine-tuned for the ecological interaction decisions in this work.

The models receive the state of a cell (species present, population amounts, environmental conditions) and a set of typed questions and return a probability distribution over the possible options. These probabilities then parameterize the population update step (reproduction, predation, migration pressure, trait adaptation), which keeps the runtime decisions cheap, deterministic in shape, and inspectable.

3.3.3 Species -> Environment: Hard Rule-Based

The reverse direction, in which species actively reshape their environment (e.g. oxygen production during the Great Oxidation Event), is deliberately **not** delegated to a learned model. It is modeled as hard, deterministic rule sets (to be defined), because:

- Environment modification is a direct physiological consequence of metabolism and population size, making it well-suited to explicit rules.
- Deterministic rules keep the environment side reproducible and prevent uncontrolled feedback loops between the learned and rule-based directions.
- Rule sets can be validated directly against geochemical records (e.g. atmospheric oxygen curves).

3.3.4 Secondary Research Question: Applicability of Generalistic Decision Models

Since these models are trained on general-purpose decision data rather than ecological domain data, their applicability in this area is an open question. As a small, non-focus subresearch question, this work examines how far such generalistic decision models transfer to ecological interaction decisions at all, and what can be inferred from their behavior—agreement with domain expectations, characteristic failure patterns, calibration drift—about the high-level biological or systems understanding such models actually possess.

3.4 Self-Supervised MARL

Note: This direction was considered as the basis for a custom reinforcement learning model, but a custom *RL* model is no longer trained in this work (see Sec. 3.3). The intrinsic-motivation concepts below remain relevant as inspiration for the typed questions posed to the decision models, and for the secondary research question on their high-level biological and systems understanding (Sec. 1.2).

Wenn biologische Organismen in eine völlig neue Umgebung geworfen werden, haben sie keinen zentralen “Score”, den sie optimieren. Sie werden durch intrinsische Reize gesteuert. In dezentralen MARL-Systemen lässt sich das abbilden, indem man Agenten nicht für ein externes Ziel belohnt, sondern für die Informationsverarbeitung.

- Curiosity (Neugier) und Vorhersagefehler: Der Agent erhält eine Belohnung (Forward Prediction Loss), wenn er Zustände mit hoher Unsicherheit erkundet oder wenn er lernt, die Konsequenzen seiner Aktionen besser vorherzusagen. Er lernt also kontinuierlich, wie die Umwelt funktioniert, wodurch sich flexible Überlebensstrategien von ganz allein entwickeln.
- Sozialer Einfluss (Social Influence): Agenten werden dafür belohnt, dass ihre Aktionen das Verhalten oder den Zustand anderer Spezies vorhersehbar beeinflussen. Dies fördert das spontane Entstehen von realistischen inter-spezifischen Rollen (wie

Prädator, Beute oder Symbiont), da die Spezies lernen, aufeinander zu reagieren, um ein soziales beziehungsweise ökologisches Gleichgewicht zu finden.

3.5 Vom trainierten Modell zur Laufzeitformel

Die in dieser Arbeit entwickelte Simulationsarchitektur trennt bewusst zwischen Training und Laufzeit. Verhaltensmodelle werden offline trainiert; zur Laufzeit liegt ihr Ergebnis nicht als neuronales Netz, sondern als explizite, evaluierbare Formel vor, die die Verhaltensphase pro Zelle auswertet. Die Softwarearchitektur unterstützt diesen Ansatz direkt: das Verhalten wird über ein Port-Interface injiziert, sodass ein trainiertes Modell und eine handgeschriebene Näherungsformel dieselbe Schnittstelle bedienen und sich gegenseitig ersetzen können, ohne dass der Simulationskern geändert werden muss.

Chapter 4

Rust Implementation

The framework is developed as a set of Rust workspace crates. Rust was chosen for its performance profile (zero-cost abstractions, no garbage collector), its expressive type system for encoding invariants, and first-class tooling for testing and benchmarking. The implementation follows a hexagonal architecture: a small kernel defines **when** simulation phases run, while injected adapters define **how** they run. This keeps the framework consumable by downstream applications (a headless CLI runner and a graphical visualization) that never need to modify kernel code.

4.1 Crate structure

The workspace separates the framework into libraries and runnable binaries:

- `pbma-model`: pure domain types with no framework dependencies. The environment state of a cell (gas amounts for O_2 , CO_2 , CH_4 and H_2O , temperature, light), the species definition with its trait values, the population quantity type with saturating non-negative arithmetic, and the global parameters (gravity, day length, axial tilt, diffusion coefficient).
- `pbma-core`: the kernel. It owns the world state, drives the tick loop, and declares the **ports** (traits) through which all simulation behavior is injected. It depends only on `pbma-model`.
- `pbma-grid`: grid cell management. Coordinates with canonical neighborhood orders, dense row-major storage with 64×64 chunk bookkeeping for later parallel work splitting, and double-buffered grids for snapshot-and-swap phases.
- `pbma-sim`: default adapters implementing the kernel ports: a mass-conserving diffusion cellular automaton, a logistic population behavior, an overcrowding-driven migration policy, and a metrics recorder.
- `runnables/pbma-cli` and `runnables/pbma-gui`: binaries that wire concrete adapters to the kernel. Only binaries may depend on concrete adapters; the libraries depend on ports exclusively.

The dependency direction is strictly inward: `model ← core ← grid/sim`, with binaries at the outside. This realizes dependency inversion at the compiler level: the kernel cannot reference adapter code even by accident.

4.2 Species registry

Species have dynamic lifetimes (speciation and extinction happen during runs), while cells are static. The species table is therefore a slotmap with generational keys: removing a species invalidates its key, and a later slot reuse bumps the generation so stale identifiers are detected instead of silently addressing the wrong lineage. Cell storage stays a plain dense vector, which is faster and simpler for a fixed grid. Populations within a cell are kept sorted by species identifier, which makes iteration order deterministic and merges cheap.

4.3 Tick pipeline

One simulation tick runs four phases in fixed order: environment step, population behavior, migration, and metrics recording. Each phase receives a read-only context for one cell: the frozen environment snapshot of the previous tick, the neighbor environments in canonical order (north, east, south, west), and the sorted populations of the cell. The kernel collects behavior deltas and migration fluxes into fresh buffers and applies them only after all computation has finished, then publishes the new environment and swaps buffers.

4.4 Determinism rules

Reproducibility is a hard requirement for the experiments of this thesis, and it is also what makes parallel execution safe. Four rules are enforced:

1. Phases read an immutable previous-tick snapshot and write to private buffers; no phase observes another phase's partial writes.
2. Neighbor aggregation always iterates in the canonical offset order.
3. Simulation logic never iterates hash maps; population lists are sorted vectors.
4. All randomness is passed explicitly through a seedable port, never generated from global state.

These rules are backed by tests: a golden-trace test asserts that the same seed and configuration produce identical metric hashes, and a property test asserts gas mass conservation under diffusion.

4.5 Population behavior and migration

The default behavior adapter implements logistic growth: each population grows toward the carrying capacity of its cell, which scales with light availability, and the growth rate is modulated by a temperature fitness curve that falls off linearly beyond a tolerance window around the species' own optimal temperature. Migration treats overcrowding as emigration pressure: when a species' local density exceeds its own crowding threshold, the excess fraction is split evenly over the four cardinal directions and emitted as migration fluxes. The kernel applies these fluxes only after all cells have been processed, so a population can never read a half-migrated neighbor.

4.6 Species registry as the behavior interface

The per-cell behavior and migration phases resolve trait values through the species registry rather than through kernel constants: growth rate, temperature optimum and tolerance, and the crowding threshold are all per-lineage quantities. This closes a feedback loop that the vertical slice could not express — two species in the same cell with different traits experience different selection pressures, and trait changes immediately change behavior without any code change.

The registry read also defines extinction semantics. When a lineage has been removed, its population entries can temporarily survive in cells until the next pruning step; such stale populations are detected by the generational key check and contribute nothing to behavior or migration. They are inert residuals, not hidden actors.

4.7 Evolution as a kernel phase

Speciation and extinction are global operations on the species table, so they sit outside the per-cell phases. A third port, the evolution engine, runs once per tick after migration and has two responsibilities. First, it reports every species that has no surviving population anywhere in the world; the kernel removes those from the registry, which invalidates their keys for every future use. Second, it spawns variant lineages of surviving species: with a configurable probability, a variant copies the parent's traits

with a multiplicative drift applied to the growth rate and a shift of the temperature optimum proportional to the parent’s tolerance width, so specialists drift less in absolute kelvin than eurytherm species. Variant names are derived deterministically from the parent name plus a batch index, and every random decision comes from an explicitly injected, seedable random number generator (a splitmix64 implementation), keeping the whole evolutionary history reproducible.

Kernel-level integration tests prove the contract: a species that loses its last populations is pruned in the same tick, stale keys are detected instead of silently aliasing the next species in the table, and a spawned variant appears in the registry with the derived trait values.

4.8 Closed-form behavior surrogate

As the first stand-in for a behavior formula derived from offline reinforcement learning, a second behavior adapter evaluates a closed-form expression per species and cell: the effective growth rate is the product of the species’ base growth rate, a gaussian temperature fitness, a saturating light response, and a quadratic crowding factor, minus a constant baseline mortality. The gaussian fitness differs from the linear ramp of the logistic adapter precisely where selection matters: mismatched species are penalized smoothly instead of being cut off at a hard tolerance boundary. A bisection test proves that the formula has an interior equilibrium where growth and mortality cancel, and that this equilibrium lies between the low-density and high-density regimes — the qualitative property a learned policy must reproduce before it can replace the surrogate.

4.9 Verification and benchmarks

The simulation is verified on three levels. Unit tests cover the domain rules of each crate, for example saturating arithmetic, registry staleness, trait-driven growth differences between coexisting species, and chunk bookkeeping. Integration tests assert the scientific invariants: a golden-trace test proves that identical seeds and configurations reproduce identical metric hashes, a conservation test proves that diffusion neither creates nor destroys gas mass, and a bookkeeping test proves that migration moves population between cells without creating or destroying it. Finally, a benchmark simulates a 1000×1000 cell world (one million cells) and measures the per-tick runtime — about 207 milliseconds per tick with the trait-resolving behavior and migration phases — providing the baseline that later parallel optimizations must improve upon.

4.10 Extensibility contract

Downstream consumers extend the framework by implementing the kernel ports rather than modifying it. A new environment model, a behavior formula derived from offline reinforcement learning, or an alternative migration policy each plug in as an adapter behind the same trait boundary; the kernel and the existing binaries remain untouched. This is the property that makes the framework a product in its own right rather than a one-off simulation script.

4.11 Diffusion cellular automaton

The default environment step implements gas diffusion as pressure equalization: for each gas, a fraction α of the positive half-difference flows from the higher to the lower concentration across each of the four neighbor pairs. Inflow and outflow are computed from the same frozen snapshot, so exchanges cancel pairwise and total gas mass is conserved exactly. α is clamped to 0.25, the stability limit of the four-neighbor scheme. Boundary cells treat the missing neighbor as a mirror of themselves, so no mass crosses the world edge.

Chapter 5

Results

5.1 Performance

5.2 Accuracy

5.3 Comparison

Chapter 6

Conclusion

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequaleamus animo, cum corpore dolemus, fieri.

6.1 Limitations

6.2 Future Work

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

Appendix A

External Resources

- GitHub Repository: <https://github.com/TimerErTim/bioinf.bachelor-thesis>
- Latorre et al. (2025) Data Repository: <https://github.com/rlatorre-uam/EcoEvoData>
- PredPreyGrass: <https://github.com/doesburg11/PredPreyGrass>

Appendix B

Source Codes

List of source codes:

B.1 Snippets

cpp

```
1 int main() {  
2     return 0;  
3 }
```

Table B.1: List of abbreviations
Appendix C

Abbreviations

Abbreviation	Description
<i>AI</i>	Artificial Intelligence
<i>TaNa</i>	Tangled Nature
<i>MARL</i>	Multi-Agent Reinforcement Learning
<i>ABM</i>	Agent-Based Modeling
<i>RL</i>	Reinforcement Learning

References

- Bosse, T., Jaffry, S. W., Siddiqui, F., & Treur, J. (2012). Comparative Analysis of Agent-Based and Population-Based Modelling in Epidemics and Economics. *Multi-Agent and Grid Systems*, 8. <https://doi.org/10.3233/MGS-2012-0193>
- HBP1969. (2026). *PredPreyGrass* (1.0.0) [Computer software]. CoMSES Computational Model Library. <https://www.comses.net/codebases/0c362544-c7cf-4498-b7d7-6ef8f6be5960/releases/1.0.0/>
- Latorre, R., Brun-Usan, M., & Fernández-Lázaro, G. (2025). Simulating macroevolutionary trends and open-ended evolution with a novel mechanistic multi-level approach. *Plos One*, 20(11). <https://doi.org/10.1371/journal.pone.0335033>
- Laurençon, H., Segerie, C.-R., Lussange, J., & Gutkin, B. (2021). *Continuous Homeostatic Reinforcement Learning for Self-Regulated Autonomous Agents*. <https://doi.org/10.48550/arXiv.2109.06580>
- Strannegård, C., Palak, M., Emgsner, N., Stocco, A., Antonelli, A., & Silvestro, D. (2025). *Predicting Ecosystem Resilience Using Multi-Agent Reinforcement Learning*. <https://doi.org/10.1101/2025.06.07.658424>

List of tables

Table B.1 List of abbreviations 22